CSE494: INTELLIGENT NOSQL DATABASES

Project report

“SyncScholar”



Submitted by:
Abhay Pratap Singh

Reg no: 12303939

Section: K4P23WS

Course code: CSE 494

Under the Guidance of

Dr. Manpreet Singh Sehgal

School of Computer Science and Engineering

Lovely Professional University, Phagwara

DECLARATION

I, **Abhay Pratap Singh**, student of **B.Tech Computer Science and Engineering**, under the CSE/IT Discipline at Lovely Professional University, Punjab, hereby declare that all the information furnished in this project report is based on my own intensive work and is genuine.

Date: 24-10-2026 Signature

Registration No: 12303939 Abhay Pratap Singh

CERTIFICATE

This is to certify that **Abhay Pratap Singh**, bearing Registration No: **12303939**, has completed **CSE 494** project titled, "**SyncScholar**" under my guidance and supervision. To the best of my knowledge, the present work is the result of his original development, effort and study.

Signature: _____

Dr. Manpreet Singh Sehgal

School of Computer Science and Engineering, Lovely Professional University

Phagwara, Punjab

Date: 24-10-2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who have supported me in completing this project titled “SyncScholar”.

First and foremost, I extend my heartfelt thanks to **Lovely Professional University** for providing me with the opportunity to undertake this project.

I am especially grateful to my project mentor and guide, Mr. Pardeep Kumar, Head of Department, School of Computer Science and Engineering, for his invaluable support, guidance, and continuous encouragement throughout this project. His knowledge and experience have been instrumental in shaping the direction of this work.

I would also like to acknowledge the open-source community, whose libraries and resources enabled me to carry out the development effectively, and the providers of the platforms used in this project, including MongoDB Atlas and Google Gemini.

Lastly, I thank my family, friends, and peers for their constant motivation, feedback, and support during every phase of the project.

Contents

1. Project Title 2
2. Problem Statement 2
3. Objectives 2
4. Executive Summary 3
5. Background and Target Users 3
6. Key User Features 4
7. System Architecture and Technology Stack 4
8. Data Design and Textbook Ingestion 5
9. AI/RAG Tutor Workflow 6
10. Assessment, Flashcard and Mastery Workflow 6
11. Offline-First/PWA Strategy and Synchronisation 7
12. Teacher Analytics Capabilities and UI Status 7
13. Setup, Configuration and Verification Scripts 7
14. Current Scope, Limitations and Items to Verify 8
15. Conclusion and Future Improvements 8

1. Project Title

SyncScholar: Curriculum-Grounded Tutoring and Formative-Practice Web Application for Low-Bandwidth Classrooms

Item	Details
Project title	SyncScholar
Course	CSE 494 Intelligent NoSql Databases
Submitted by	Abhay Pratap Singh (Reg. No. 12303939, Section K4P23WS)
Guide	Dr. Manpreet Singh Sehgal
Aligned goal	UN Sustainable Development Goal 4 (Quality Education)
Basis of report	Review of the SyncScholar repository brief (source-code review only)
Intended audience	Academic evaluators, project reviewers, educators and software developers

2. Problem Statement

Students in underserved and multi-grade classrooms often face unreliable or expensive internet access and have limited access to individual tutoring. A single classroom may contain learners at different grade levels, each needing help that follows their assigned textbook and course level. At the same time, teachers need an efficient way to identify the learning gaps that are common across their students. Tools that depend on constant connectivity, or that are not tied to a specific curriculum, may not serve these classrooms well.

Problem statement: How can curriculum-grounded tutoring and formative practice be delivered to students in low-bandwidth or intermittently connected classrooms, in a way that supports different grade levels, keeps student progress safe while offline, and gives teachers visibility into learning gaps?

3. Objectives

The overall objective of SyncScholar is to make curriculum-focused help and practice more accessible to students in low-connectivity, multi-grade settings, in line with UN Sustainable Development Goal 4. The specific objectives, together with their implementation status according to the source review, are listed below.

Objective	Status in the current repository
Provide Socratic, textbook-grounded tutoring with citations	Implemented in source: retrieval-augmented tutor flow with follow-up questions and citation cards. Live curriculum coverage is unverified.
Offer practice quizzes and flashcards	Implemented in source; the assessment response shape between server and client needs to be verified.
Keep core learning functions usable offline	PWA shell caching, local IndexedDB storage and a sync queue are implemented. Live AI tutoring is not available offline.
Support different grade levels in one classroom	Grade and subject filters are implemented; below 0.50 mastery the tutor also searches the preceding grade.
Track concept-level mastery and guide remediation	Mastery tracking with an exponential moving average is implemented; a full prerequisite graph is not.
Give teachers insight into class-wide learning gaps	Backend analytics APIs are implemented; no teacher dashboard exists in the client yet.

4. Executive Summary

SyncScholar is an educational tutoring and formative-practice web application designed for students in low-bandwidth or intermittently connected classrooms. Its goal is to make curriculum-focused help and practice more accessible, particularly in underserved and multi-grade settings, and its stated mission aligns with UN Sustainable Development Goal 4 (Quality Education).

The repository is organised as a monorepo with three technical parts: a React/Vite student-facing Progressive Web App (PWA); a Node.js/Express API backed by MongoDB Atlas and Google Gemini services; and a separate Node.js curriculum-ingestion workspace that prepares textbook chunks, creates embeddings and writes them to MongoDB.

The source code implements a substantial student learning workflow: a curriculum question-and-answer tutor, practice quizzes and flashcards, student sign-in and a local classroom mode, a mastery summary, cached learning materials, and a local queue for quiz attempts that synchronises when connectivity returns. The backend also provides teacher analytics APIs; however, no teacher dashboard exists in the current React client.

This report distinguishes implemented code from design aspirations and lists integration items that should be verified before the system is described as production-ready.

5. Background and Target Users

5.1 The educational problem

The project addresses barriers faced by students who may have:

- Unreliable or expensive internet access.
- Limited access to individual tutoring.
- Different grade levels within the same classroom.
- A need for help that follows their assigned textbook and course level.
- Teachers who need an efficient way to identify common learning gaps.

5.2 Teaching approach

The intended approach is supportive and Socratic: the tutor offers conceptual guidance, explanations, examples and a next-step question rather than acting as a simple answer key. Textbook citations are intended to let students and teachers trace explanations back to source material.

5.3 Target users

User	Role in the system
Students	Ask curriculum questions, take practice quizzes, review flashcards and view their mastery profile. Fully served by the current frontend.
Teachers	Intended to review classroom overviews, weak-spot heatmaps and struggling-student lists. Currently supported by backend APIs only.
Administrators	Role exists in the user model and authorization middleware.

6. Key User Features

6.1 Student-facing features (implemented in the frontend)

- **Curriculum Doubt Tutor:** subject and grade selectors, chat-style questions, suggested questions, responses, follow-up prompts and citation cards.
- **Practice Quiz & Flashcards:** topic selection, multiple-choice practice, explanations, a score, flashcard review and local caching.
- **Student mastery modal:** requests a cloud profile and shows concept scores, progress status, recent quiz activity and remediation indicators when data is available.
- **Authentication options:** Clerk sign-in/sign-up when configured, school roll-number login through the API, and a local offline classroom guest mode.
- **Connectivity and sync status:** a persistent header indicator with an action to trigger synchronisation when attempts are queued.

The interface uses React components, Tailwind CSS, Lucide icons, high-contrast cards, focus styles, responsive layouts and minimum touch-target sizing.

7. System Architecture and Technology Stack

The system is a monorepo with separate package manifests and lockfiles under client/, server/ and ingestion/, plus docs/ and Prompt/ folders. There is no root package manifest.

Component	Description
Frontend (client/)	Vite and React 18; Tailwind CSS; vite-plugin-pwa/Workbox service worker that precaches the application shell and SVG assets and applies a network-only rule to API health checks; Dexie.js (IndexedDB) for cached flashcards, quizzes, tutor history, the current session and pending submissions. services/api.js wraps fetch with timeouts, auth headers and network error messages; services/syncService.js monitors connectivity and backend health and uploads queued attempts in batches.
Backend (server/)	Express REST API; Mongoose models for users, textbook chunks and quiz attempts; MongoDB Atlas as system of record; Google @google/genai SDK for embeddings and generated tutor/assessment responses; local JWT and Clerk token verification with role-based authorization. Code is divided into config, controllers, routes, services, models, middleware and utilities.
Ingestion (ingestion/)	A separate Node.js package that extracts or processes textbook content, divides it into educational chunks, attaches curriculum metadata, creates 768-dimensional Gemini embeddings and upserts documents to MongoDB Atlas.
Documentation (docs/)	Architecture notes on collections, vector retrieval, mastery tracking, offline synchronisation and teacher analytics; Atlas vector index definitions.
Prompt templates (Prompt/)	Implementation instructions and design history for adaptive RAG, authentication, ingestion, offline PWA, student UI, flashcards and teacher analytics. These are not proof that every proposed capability is complete.

7.1 API surface

API payloads are JSON. The client uses VITE_API_BASE_URL or the default “/api”; during Vite development “/api” is proxied to localhost:5000.

Route group	Purpose
/api/auth	Registration, login and current-user lookup

/api/student	Profile/mastery and grade update
/api/tutor	Curriculum tutor question
/api/assessments	Generate, submit, batch-sync and quiz history
/api/teacher	Classroom overview, heatmap and objective-level student drill-down
/api/health	Server and MongoDB connection status

8. Data Design and Textbook Ingestion

8.1 MongoDB collections

- **users** (server/src/models/User.js): student, teacher and admin roles; email, school roll number or Clerk identity; school and grade; teacher-assigned grades; preferred language; and a concept_mastery array keyed by learning objective with mastery score, attempts, status and last-reviewed date.
- **curriculum_chunks** (server/src/models/CurriculumChunk.js): text passage and 768-number embedding, with metadata for source curriculum, grade, subject, chapter, learning objective, difficulty tier, page and official source URL. The Atlas Vector Search index uses the “embedding” field with cosine similarity and filters on grade, subject, learning objective and difficulty.
- **quiz_attempts** (server/src/models/QuizAttempt.js): student and client-generated attempt ID, objective, subject, grade, evaluated answers, score, maximum score, misconception tags and an offline-sync flag. A unique compound student/attempt ID index supports deduplication.

8.2 Ingestion process

The ingestion workspace is implemented in JavaScript/Node.js (some project guidance describes a Python pipeline, which does not match the code). Its scripts cover PDF ingestion, extracted-text ingestion, sample seeding, supplementary seeding and database/vector verification. Raw PDF files are excluded from Git; the repository includes an extracted Class 9 Science text file, sample metadata and the scripts.

8.3 Curriculum coverage

Seed examples and offline fallback decks focus mainly on Class 7 Science (Heat) and Class 9 Science (Sound). The extracted-text pipeline covers NCERT Class 9 Science chapter material, and supplementary seeds include some Class 8 and legacy Class 9 topics. A manifest also references Class 9 Mathematics PDFs, but their presence or ingestion into a live database cannot be verified from the repository alone. The repository does not establish which chunks exist in any live Atlas database, so no chunk counts or completeness claims are made.

9. AI/RAG Tutor Workflow

The tutor uses retrieval-augmented generation (RAG) to ground answers in textbook passages. The documented and implemented flow is:

1. The student submits a question, subject, grade and recent conversation turns from the Tutor view.
2. The API validates the request and identifies the student context.
3. An adaptive filter uses the student's mastery data, grade and subject to build a MongoDB Vector Search pre-filter.
4. If mastery is below 0.50, the filter expands to include the immediately preceding grade as well as the current grade.
5. Gemini (gemini-embedding-001, 768 dimensions) embeds the query and Atlas searches the curriculum chunks using the "vector_index" index with a small retrieval limit.
6. A relevance threshold (currently 0.72) can refuse a query without calling the answer model.
7. Otherwise, retrieved passages and conversation context are sent to a Gemini text model instructed to give a grade-appropriate Socratic response.
8. The response includes a follow-up question and citation metadata. The frontend formats mathematical notation and caches tutor exchanges for later reading.

Tutor and assessment generation use configurable Gemini model names with fallbacks. Prompt instructions emphasise textbook grounding, Socratic explanation, grade-appropriate language and structured JSON output.

10. Assessment, Flashcard and Mastery Workflow

1. The API retrieves curriculum chunks (first by exact learning-objective match, then vector retrieval, then grade/subject chunks) and requests three diagnostic multiple-choice questions and three flashcards in JSON.
2. The backend sanitises the generated assessment and returns it with source references.
3. The student answers; the browser calculates an immediate score and writes the attempt to Dexie before attempting network submission.
4. If online, the frontend submits promptly. If the request fails or the student is offline, the pending item stays in IndexedDB for retry.
5. The backend records the attempt using the client-generated ID, evaluates answers, tracks incorrect concept tags and updates mastery with an exponential moving average (EMA).
6. The sync service batches up to 50 pending attempts, filters them to the active student and retries when connectivity returns. The server checks the attempt ID so duplicates are not re-scored.

Mastery rules

Parameter / band	Current implementation
EMA alpha	0.4
Below 0.50	Needs prerequisite remediation
0.50 – 0.79	Progressing
0.80 or above	Mastered

11. Offline-First/PWA Strategy and Synchronisation

Offline support has several distinct layers:

- **Application shell:** the service worker caches the frontend shell and static assets so the app can load offline after initial access.
- **Local data:** Dexie/IndexedDB stores cached quizzes, flashcards, tutor history, student session information and unsynced attempts.
- **Status indicators:** the UI signals online, offline, pending and syncing states.
- **Fallback content:** two starter assessment decks and several hardcoded tutor-answer patterns provide limited local practice and help without a live RAG request.
- **Health monitoring:** a periodic backend health check supplements browser online/offline events.
- **Retry:** pending quiz attempts are retried by the sync service after network recovery.

Important limitation. Offline mode does not provide live LLM tutoring or full curriculum retrieval without the server and database. The embedded fallback content is limited, and wider offline coverage depends on previously cached data.

12. Teacher Analytics Capabilities and UI Status

The Express API provides three teacher endpoints: a classroom overview, a weak-spot heatmap, and a struggling-student drill-down. They aggregate student mastery by learning objective and grade, compute average mastery and struggling counts, look up quiz misconception tags, and list students below the 0.50 mastery threshold for an objective. The endpoints require teacher or admin roles and restrict results to the school and assigned-grade context.

Current UI status. No teacher page, dashboard component or navigation for these APIs was found in the current client source. Teacher analytics is therefore a backend capability only at present.

13. Setup, Configuration and Verification Scripts

Each workspace has its own environment template. The server expects a MongoDB URI, a Gemini API key and a JWT secret; Clerk keys are optional and configuration-dependent. Actual credentials must never be committed or included in reports.

Workspace	Scripts
client	npm run dev, npm run build, npm run preview
server	npm run dev, npm start (the “test” script is a placeholder that exits with an error)
ingestion	npm run seed, npm run ingest, npm run ingest-science, npm run dry-run, npm run verify

Additional verification scripts exist under server/scripts and ingestion/scripts; many require a configured MongoDB database and/or Gemini credentials. This report does not state that any tests or a production deployment have passed, because no such results were available.

14. Current Scope, Limitations and Items to Verify

The following observations come from source review, not a full runtime test.

Item	Observation
Assessment response shape	assessmentService.js returns “diagnostic_mcqs” and “study_flashcards”, while AssessmentView.jsx reads response.data.questions and response.data.flashcards. An adapter should be located, or the mismatch fixed, before claiming that freshly generated online quizzes populate correctly.
Teacher UI	Backend endpoints and role checks exist, but the React app has no teacher dashboard.
Readability	Prompts request grade-appropriate wording, but no active Flesch-Kincaid scoring was found; reading level is not measured.
Prerequisite scope	Design documents describe a prerequisite graph; the adaptive filter only expands to the previous grade plus current grade and does not traverse prerequisite learning objectives.
Curriculum availability	Scripts and samples show intended coverage but do not establish live database contents.
Ingestion language	Code is Node.js; some guidance describes Python.
Database naming	Runtime and most ingestion defaults use “syncscholar”; docs/atlas_vector_index.json names “edubridge” as target database. The intended deployment database should be clarified.
Operational status	MongoDB Atlas, Gemini and the running application were not exercised; no hosted deployment was reviewed.

Historical QA context

FAILURE_REPORT.txt, dated 2026-11-02, records an NCERT Class 9 Science evaluation of 70 test cases that reported 11 bugs. It describes an earlier state of the code—for example, it reports a 0.78 relevance threshold and missing conversation history, whereas current source shows a 0.72 threshold and conversation-history handling. It should be read only as historical testing context limited to that NCERT Class 9 Science scope, not as evidence that its findings or metrics describe the current code.

15. Conclusion and Future Improvements

SyncScholar combines retrieval-augmented tutoring, diagnostic practice, mastery tracking and an offline-first architecture to address barriers to learning in low-connectivity classrooms. The source code implements a coherent student workflow and a teacher analytics backend, with a deliberate design for local caching and deduplicated synchronisation. Its current maturity should be described as a code-complete prototype under review rather than a validated, deployed system.

Practical next steps suggested by the review:

- Resolve the assessment response-shape mismatch and add end-to-end tests for quiz generation, submission and sync.
- Build a teacher dashboard in the React client on top of the existing analytics APIs.
- Implement a true prerequisite graph for remediation rather than a previous-grade filter.
- Add measurable readability checks (for example, Flesch-Kincaid) if reading-level claims are to be made.

- Expand and verify curriculum coverage, including Class 9 Mathematics, and document live database contents.
- Broaden offline fallback content and replace the placeholder server test script with automated tests.
- Re-run the NCERT Class 9 Science evaluation against current code, and pilot with teachers and students to gather real outcome evidence.
- Reconcile documentation inconsistencies (Python vs Node.js ingestion; “edubridge” vs “syncscholar”).